

RB Tree Construction

1 | Concepts

A Red Black Tree is a Binary Search Tree with the following additional properties

- Every Node is either **RED** or **BLACK** (**Coloring Property**)
- Each **NULL** pointer is Considered to be a **BLACK** Node
- If a node is **RED**, then both of its Children are **BLACK** (**RED Node Property**)
- Every path from a Node to a Leaf Node contains the same number of **BLACK** Nodes (**BLACK Height Property**)
- Root Node is **BLACK** (**ROOT Node Property**)

1.1 | Benefit

The main benefit of a RED - BLACK Tree is that it provides a good worst-case runtime for its operations

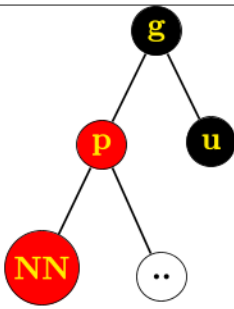
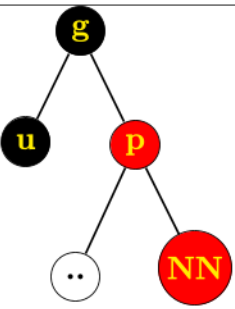
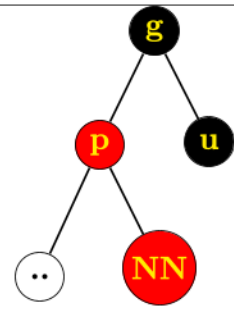
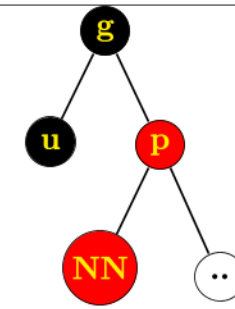
- It is a **Self Balancing Tree**
- The Height of the tree remains **logarithmic** in the number of nodes
- Ensure that the **Insert / Delete / Search** operations run in **O(Log n)** time where **n** is the number of nodes in the tree
- **Operations are similar to those of BST** but with additional steps to adjust colors and perform rotations to maintain the tree's Balancing properties

1.2 | Algorithm

Algorithm 1 Insert Operation in Red-Black Tree

```
1: Create a NewNode (NN) and Color it RED
2: Insert into RBT using BST Principle
3: while Color(Parent(NN)) is RED do
4:   if Color(Uncle(NN)) is RED then
5:     Color(Parent(NN)) = Color(Uncle(NN)) = BLACK
6:     Color(GrandParent(NN)) = RED
7:     GrandParent(NN) is now treated as NN
8:   else
9:     // if the Color(Uncle(NN)) is BLACK, then perform rotations to balance the tree
10:    next section
11:   end if
12: end while
13: Color(Root Node) = BLACK
```

if color(Uncle(NN)) is BLACK then any one of the following 4 Scenarios arise

LEFT-LEFT Case	RIGHT-RIGHT Case	LEFT-RIGHT Case	RIGHT-LEFT Case
			
Swap Colors of p and g Right Rotate on g	Swap Colors of p and g Left Rotate on g	Left Rotate on p Swap Colors of NN and g Right Rotate on g	Right Rotate on p Swap Colors of NN and g Left Rotate on g

The rotation Concepts remain the same as in AVL Trees !!!

MCA Sem I - 2024 - 25

©Dilip Maripuri

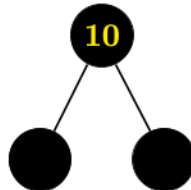
2 | Illustration : 10,18,7,15,16,30,25,40,60,2,1,70

2.1 | Element : 10

Create a Rednode with 10 in it

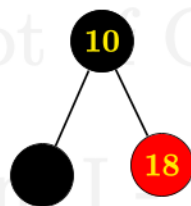


since 10 is a root node, Color it Black



2.2 | Element : 18

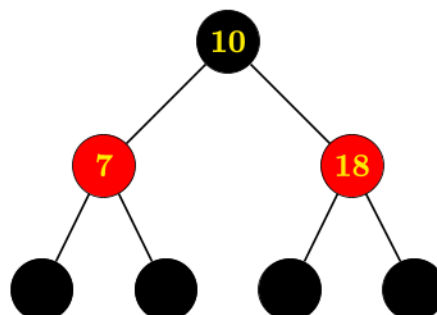
Create a Rednode with 18 in it and insert



No Violation

2.3 | Element : 7

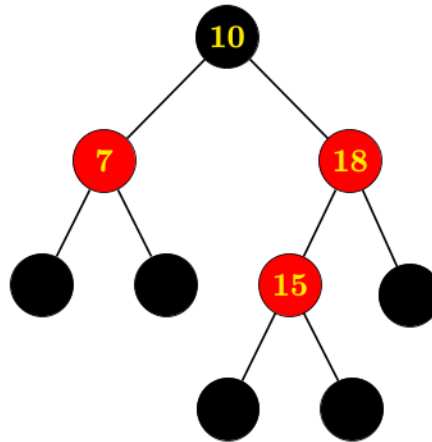
Create a Rednode with 7 in it and insert



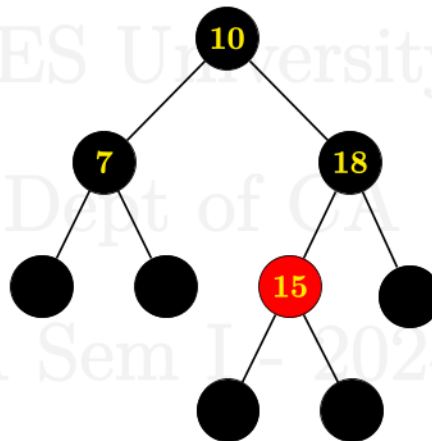
No Violation

2.4 | Element : 15

Create a Rednode with 15 in it and insert



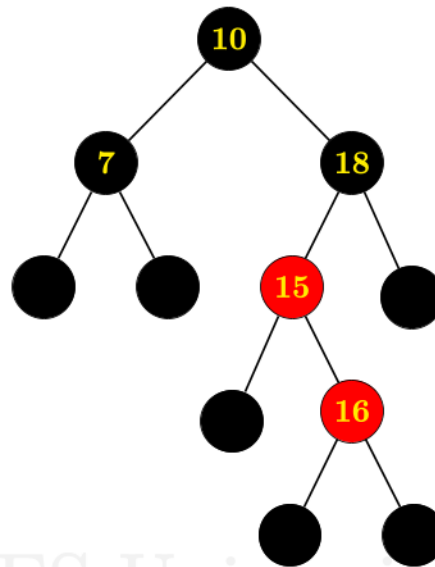
Violation :



1. BLACK Height Property : **Satisfied**
2. RED Node Property : **Satisfied**
3. ROOT Node Property : **Satisfied**

2.5 | Element : 16

Create a Rednode with 16 in it and insert



Violation : **RED Node Property Violation !!!**

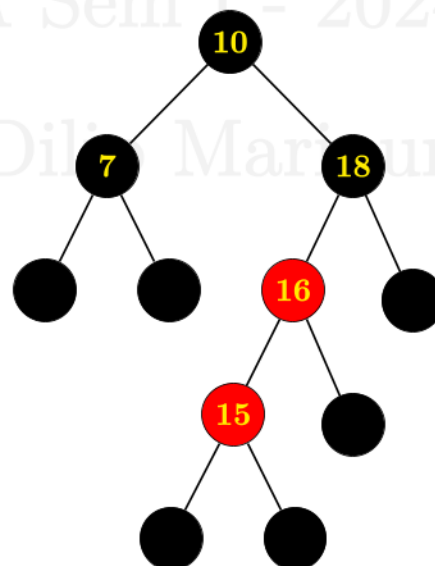
Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? TRUE

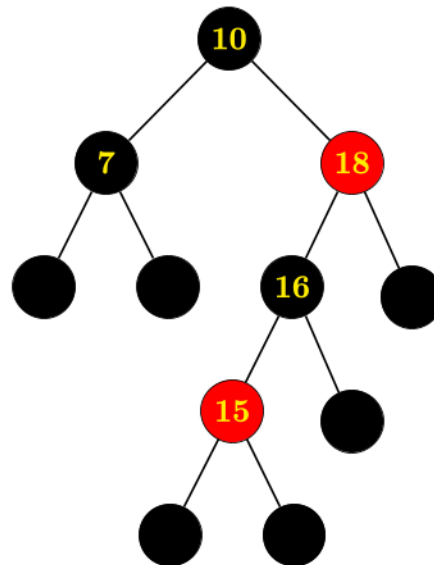
Rotation to be performed !! :

Left - Right Case (Left on 15, Swap Colors of G, NN , Right on 18)

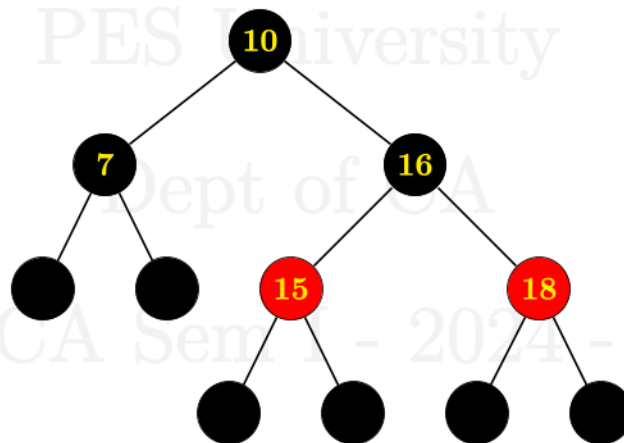
Left Rotate on 15



Swap Colors of 18 and 16



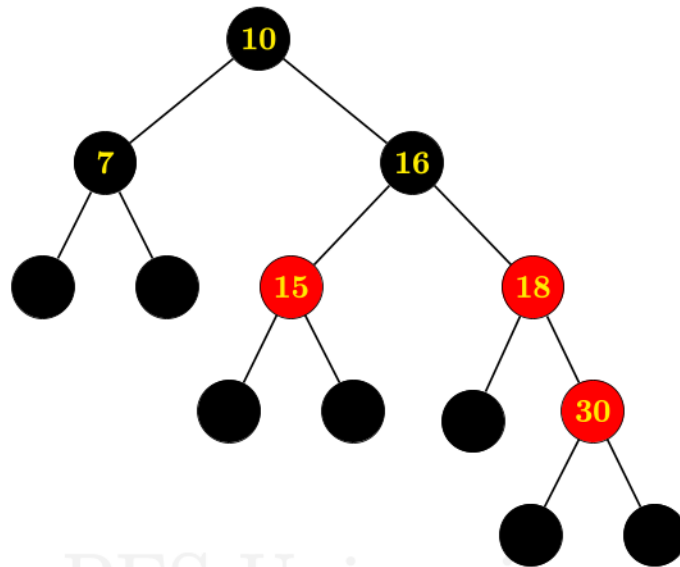
Right Rotate on 18



1. BLACK Height Property : **Satisfied**
2. RED Node Property : **Satisfied**
3. ROOT Node Property : **Satisfied**

2.6 | Element : 30

Create a Rednode with 30 in it and insert

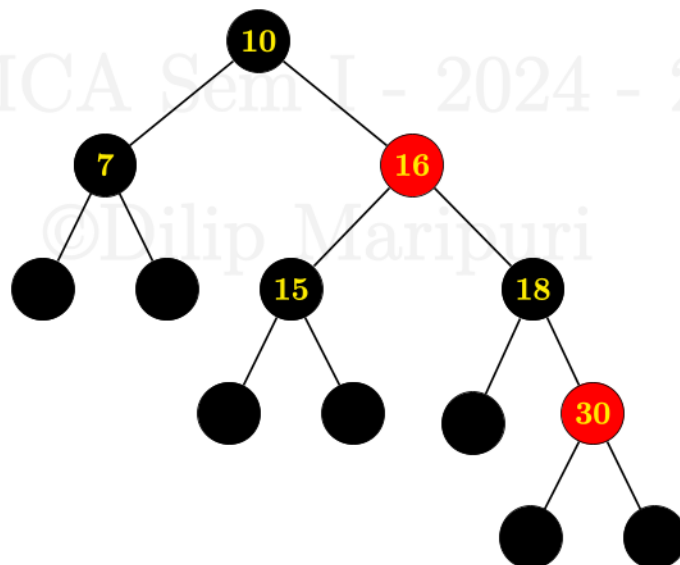


Violation : **RED Node Property Violation !!!**

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? FALSE

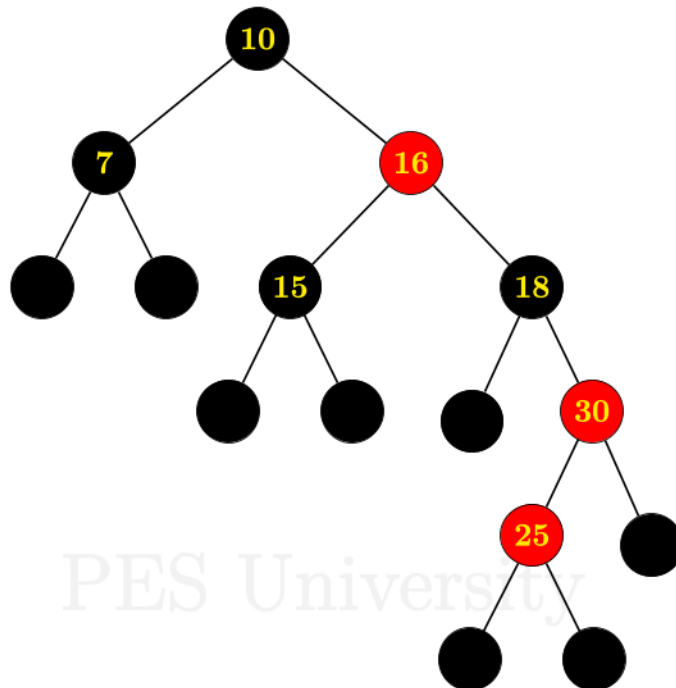
Recolor (15,18 BLACK)(16 RED) Parent(16 Already Black !!!)



1. BLACK Height Property : **Satisfied**
2. RED Node Property : **Satisfied**
3. ROOT Node Property : **Satisfied**

2.7 | Element : 25

Create a Rednode with 25 in it and insert



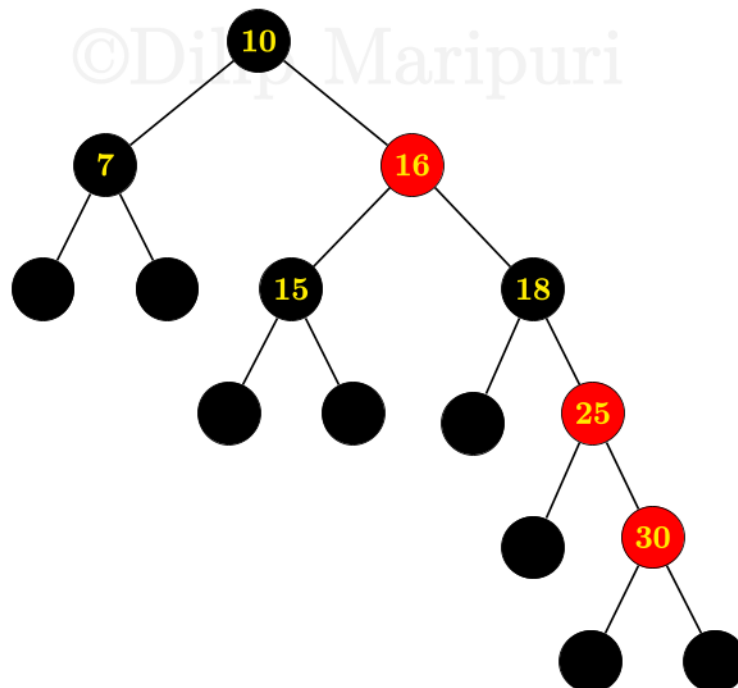
Violation : **RED Node Property Violation !!!**

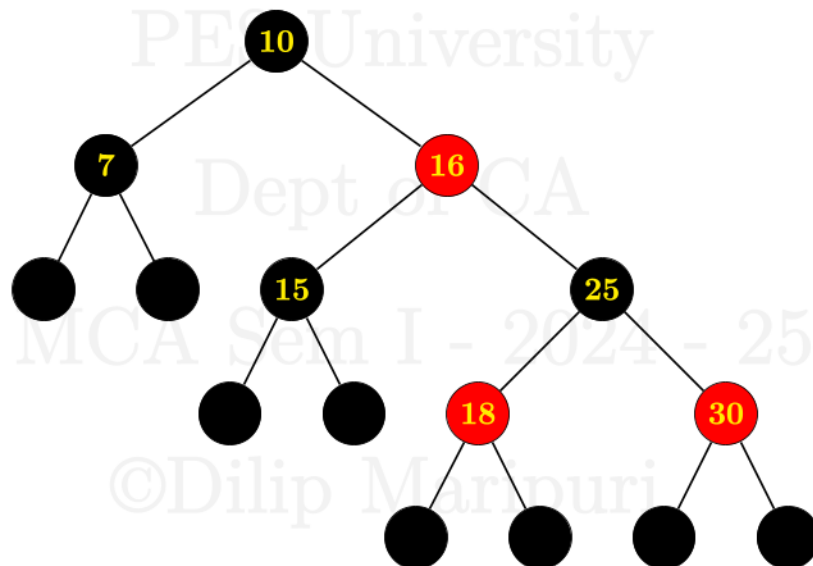
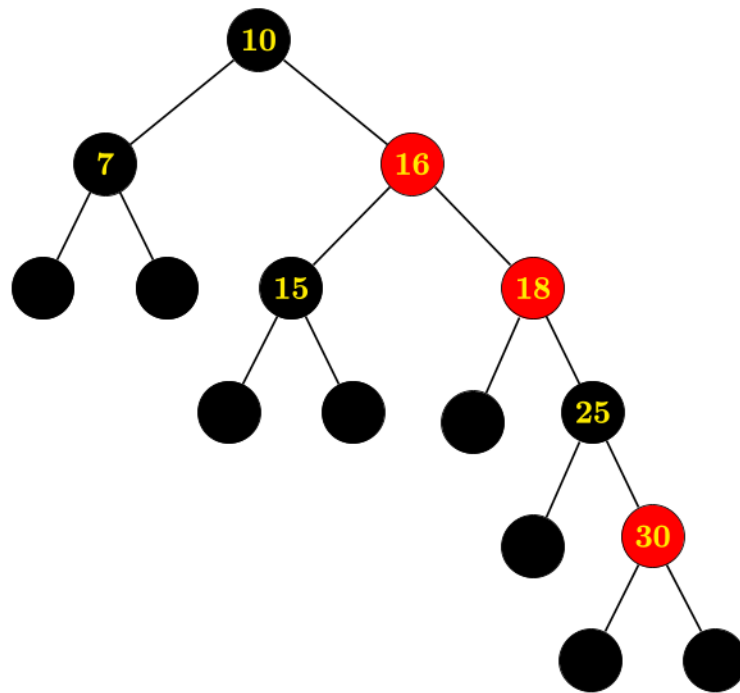
Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? TRUE

Rotation to be performed !!

Right - Left Case (Right on 30, Swap Colors of G, NN , left on 18)

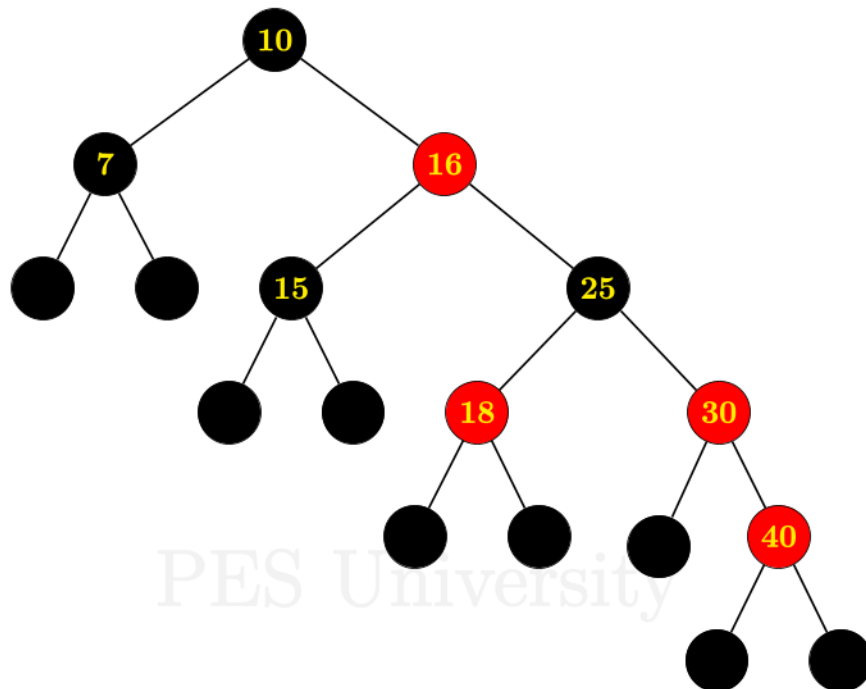




1. BLACK Height Property : **Satisfied**
2. RED Node Property : **Satisfied**
3. ROOT Node Property : **Satisfied**

2.8 | Element : 40

Create a Rednode with 40 in it and insert

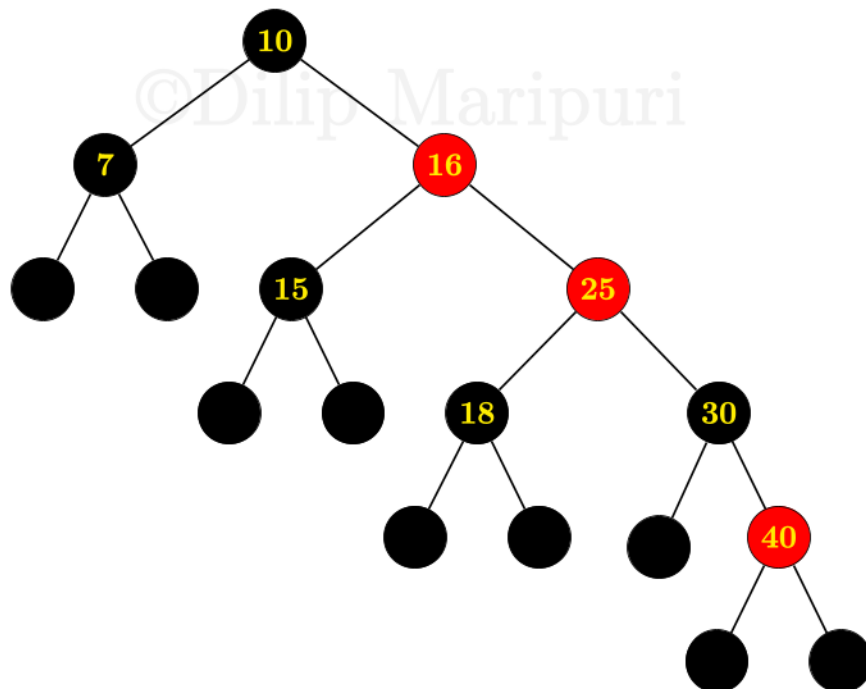


Violation : **RED Node Property Violation !!!**

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? FALSE

Recolor (18,30 BLACK)(25 RED); 25 is new node

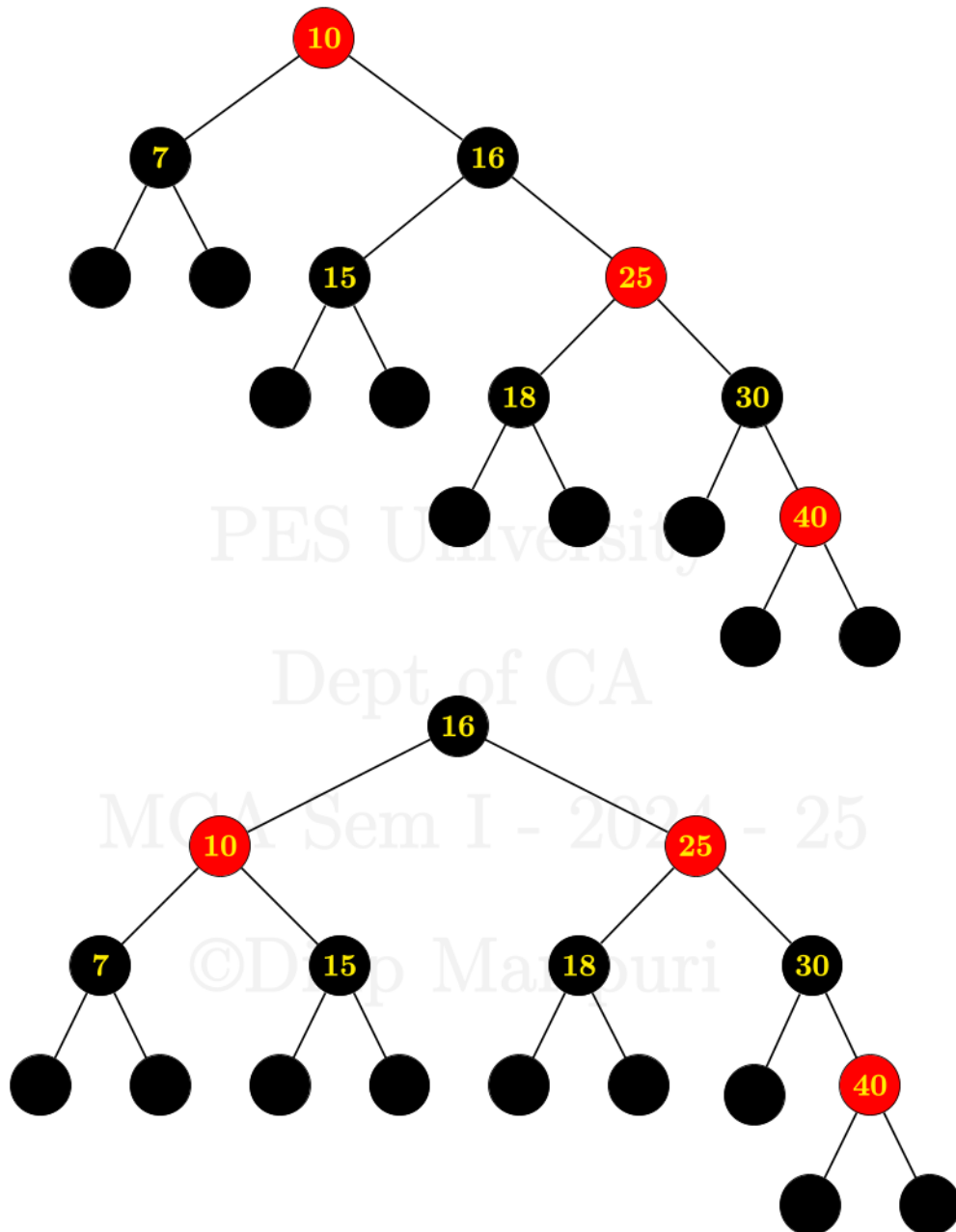


Violation : **RED Node Property Violation !!!** (New Node : 25)

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? TRUE

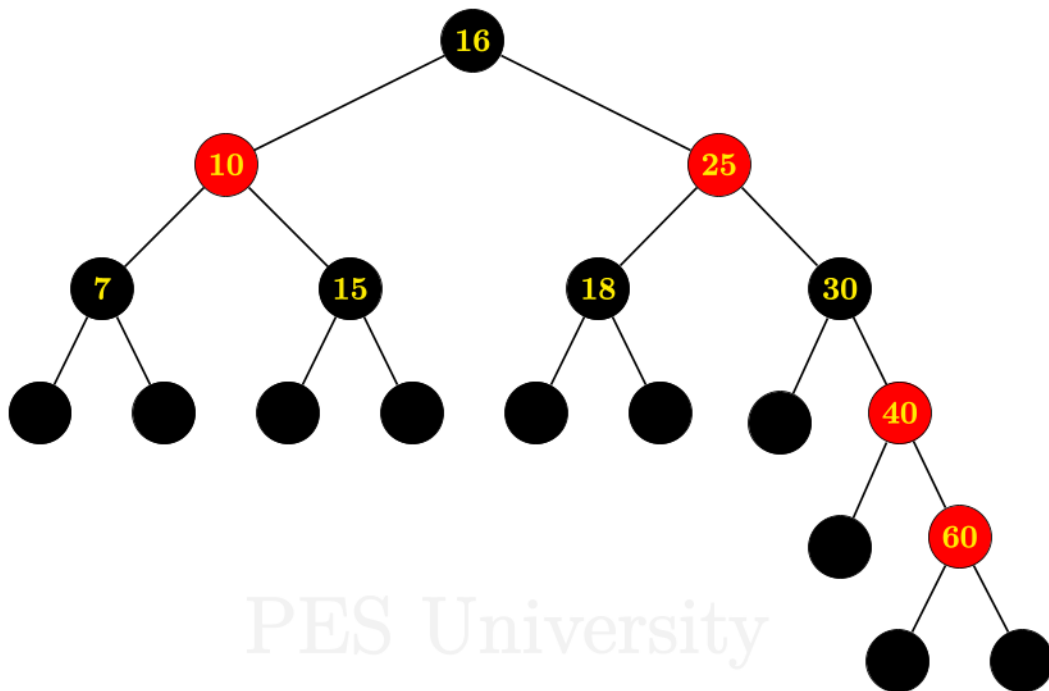
Swap Colors of 10,16 and left rotate on 10



1. BLACK Height Property : **Satisfied**
2. RED Node Property : **Satisfied**
3. ROOT Node Property : **Satisfied**

2.9 | Element : 60

Create a Rednode with 60 in it and insert

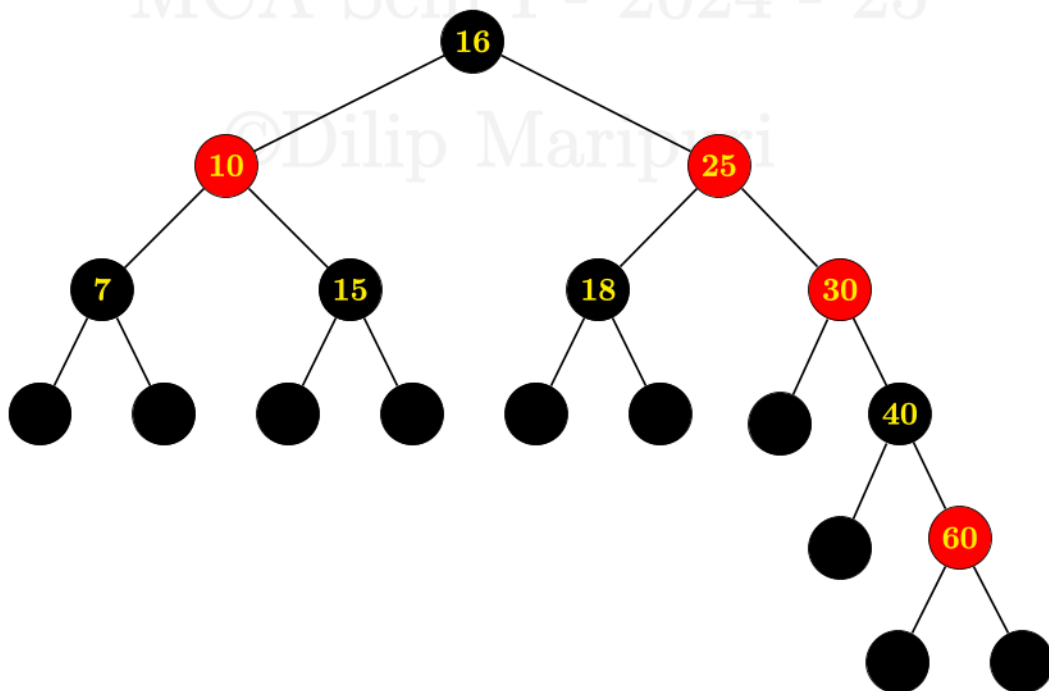


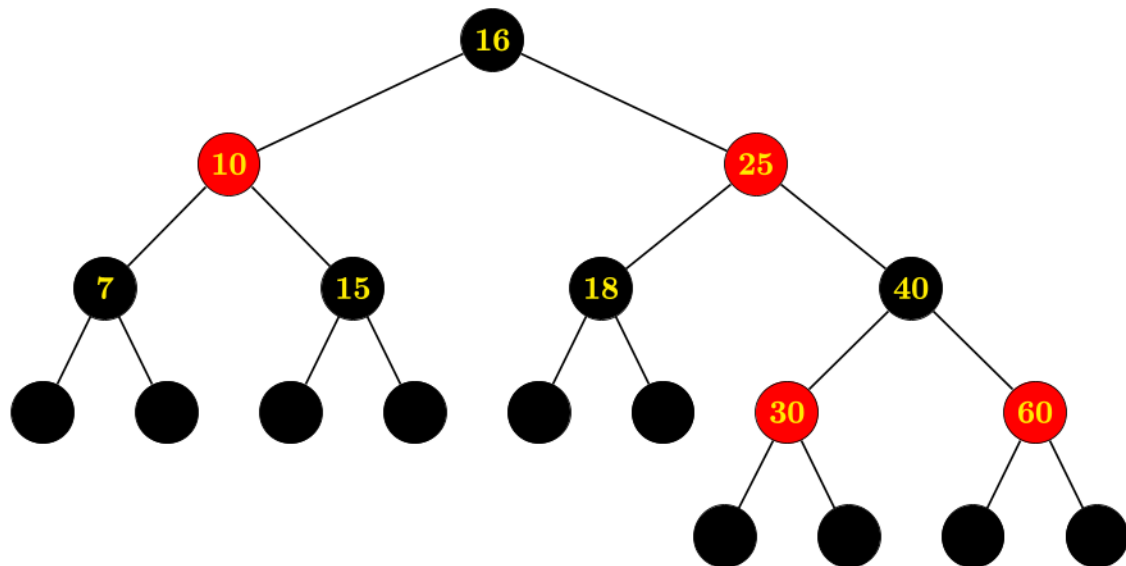
Violation : **RED Node Property Violation !!!**

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? No Uncle (NULL Pointer is Black)

Swap Colors(30,40) Rotate to left on 30

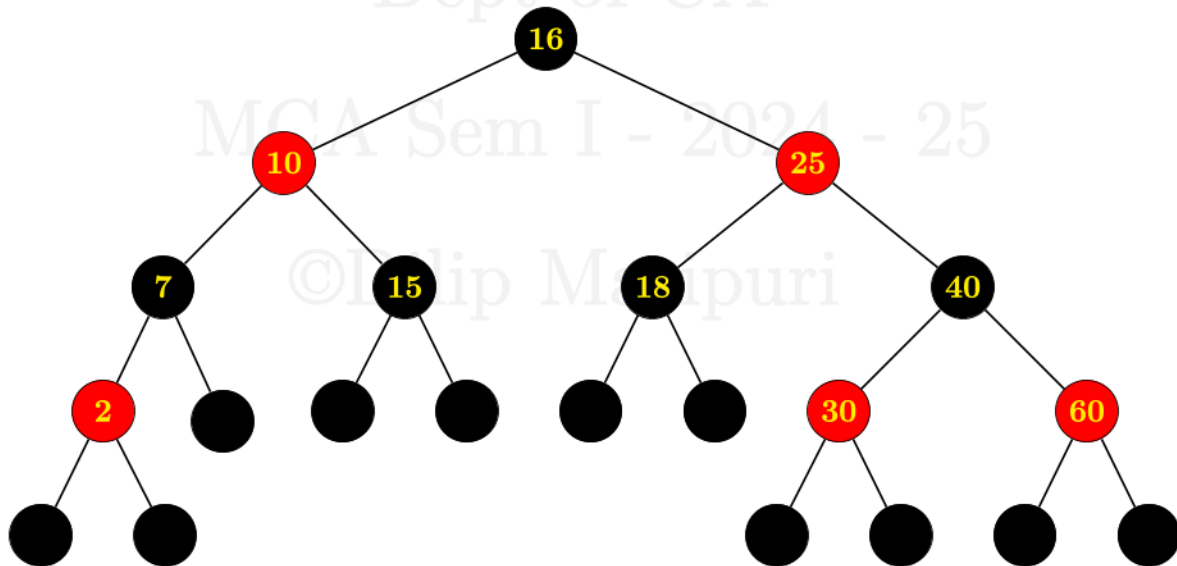




1. BLACK Height Property : Satisfied
2. RED Node Property : Satisfied
3. ROOT Node Property : Satisfied

2.10 | Element : 2

Create a Rednode with 2 in it and insert

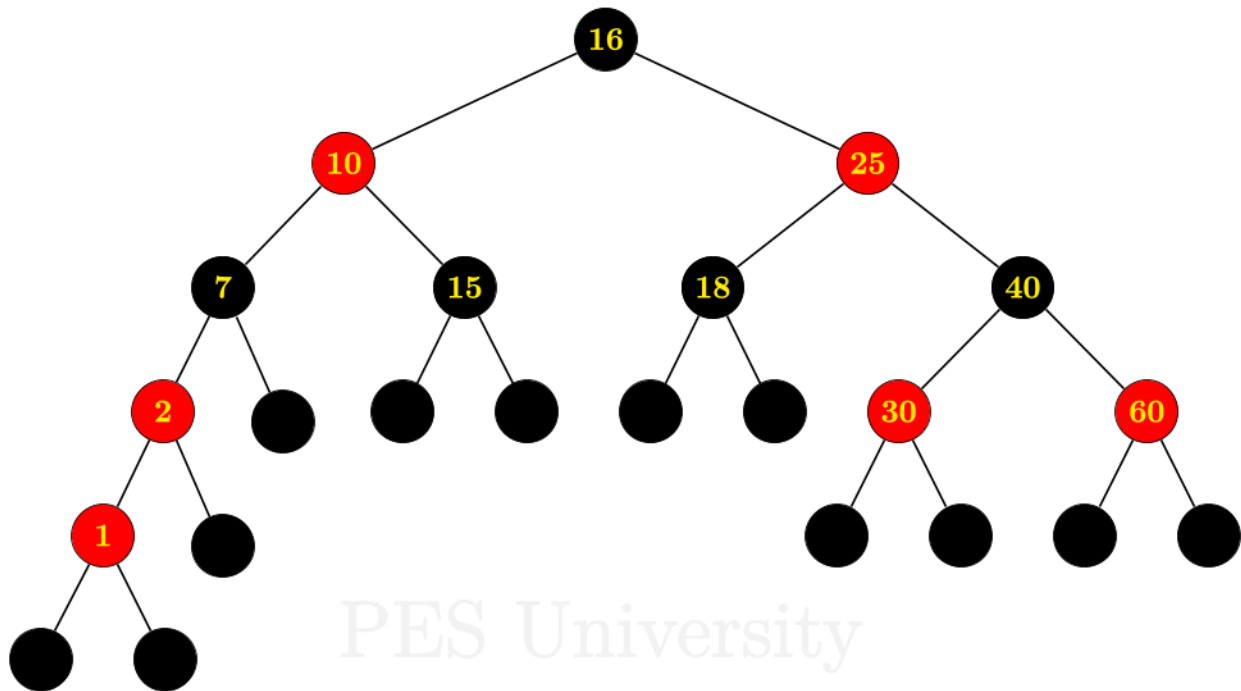


No Violation !!

1. BLACK Height Property : Satisfied
2. RED Node Property : Satisfied
3. ROOT Node Property : Satisfied

2.11 | Element : 1

Create a Rednode with 1 in it and insert

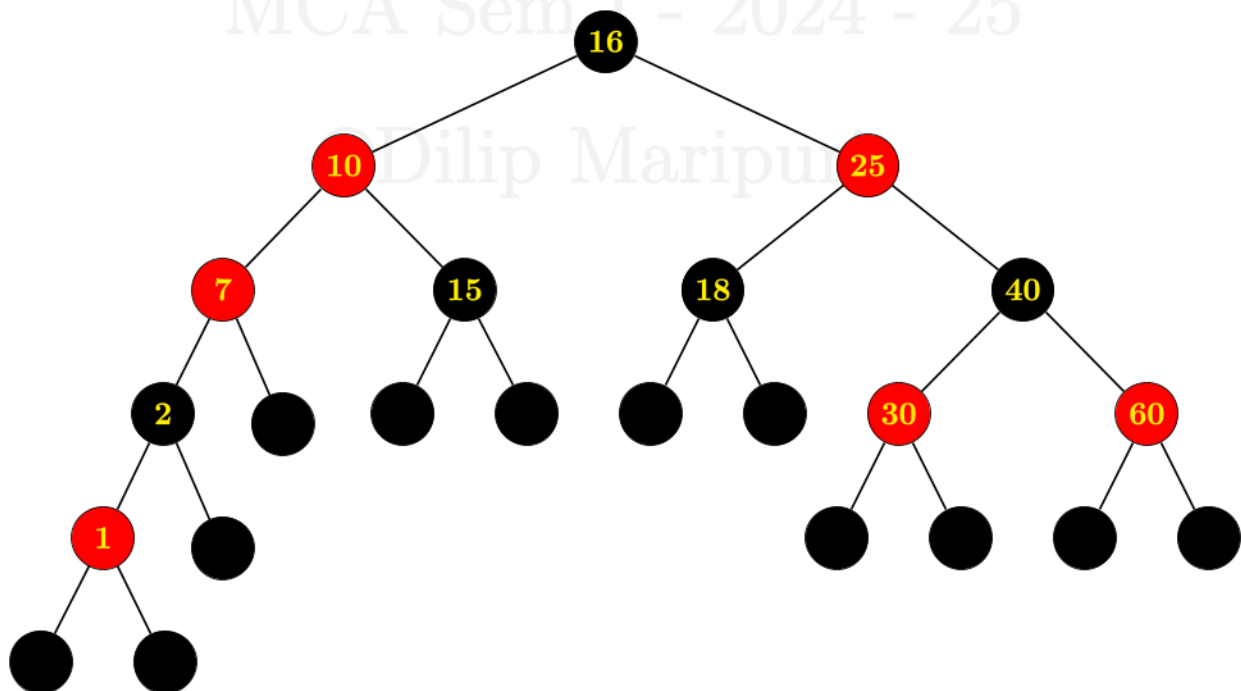


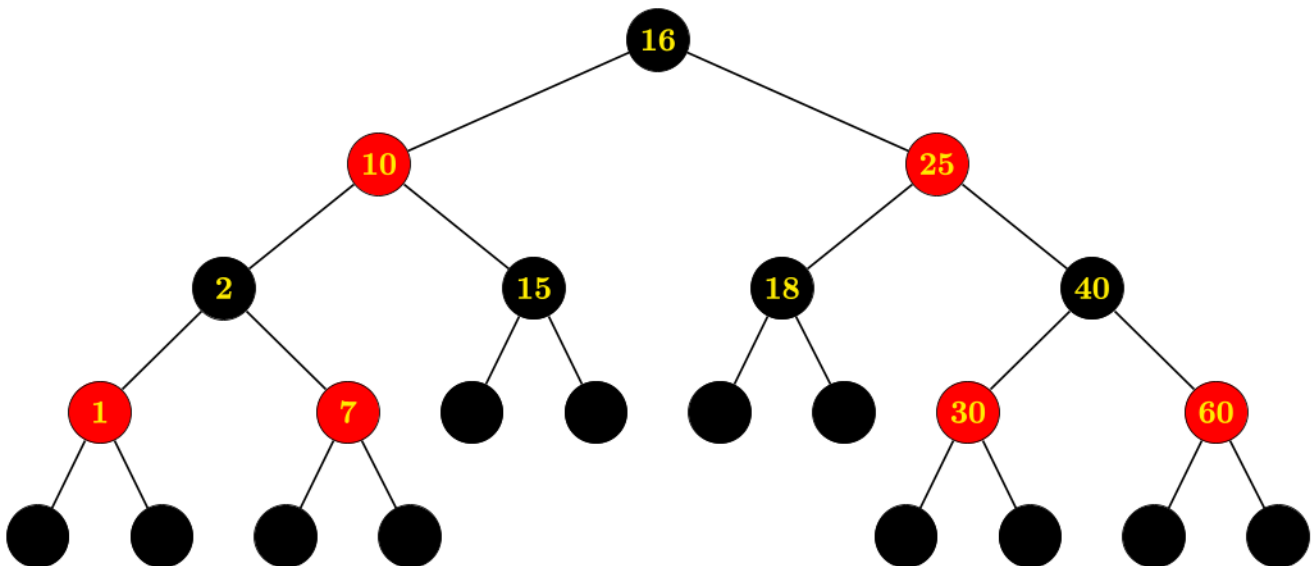
Violation : **RED Node Property Violation !!!**

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? No Uncle (NULL Pointer is Black)

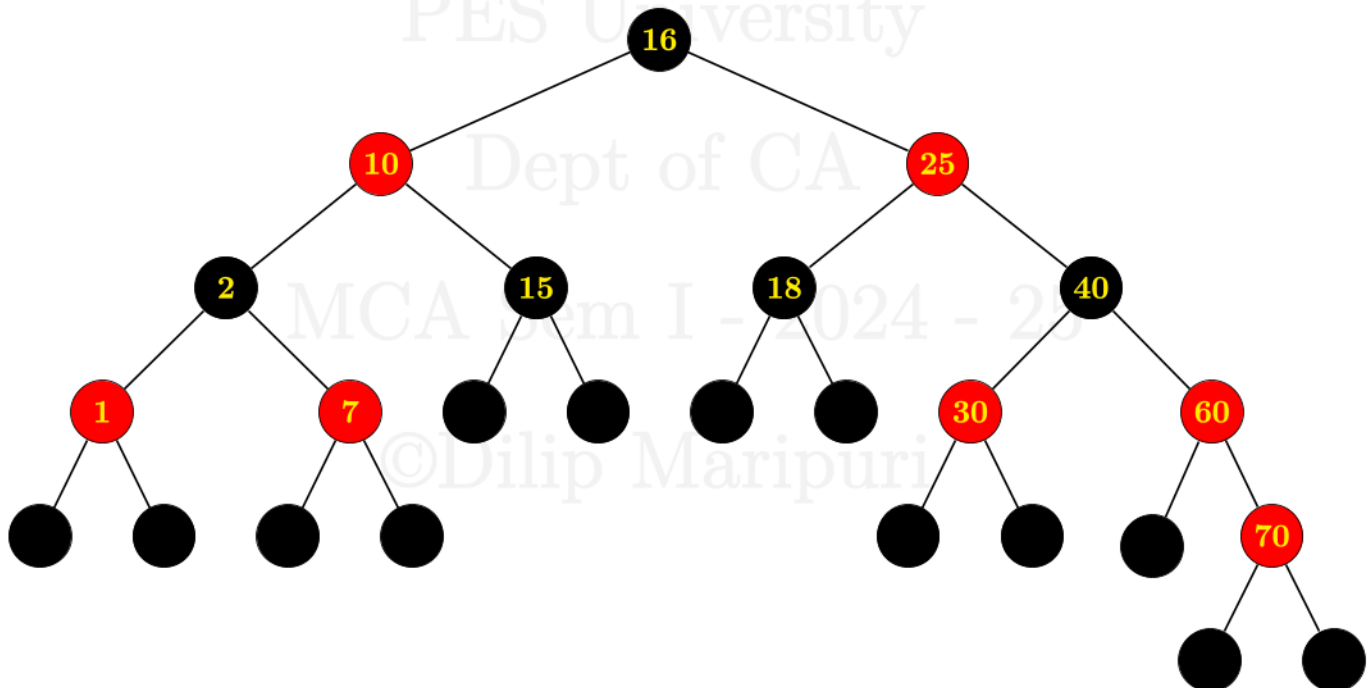
Swap Colors(2,7) Rotate to Right on 7





2.12 | Element : 70

Create a Rednode with 70 in it and insert

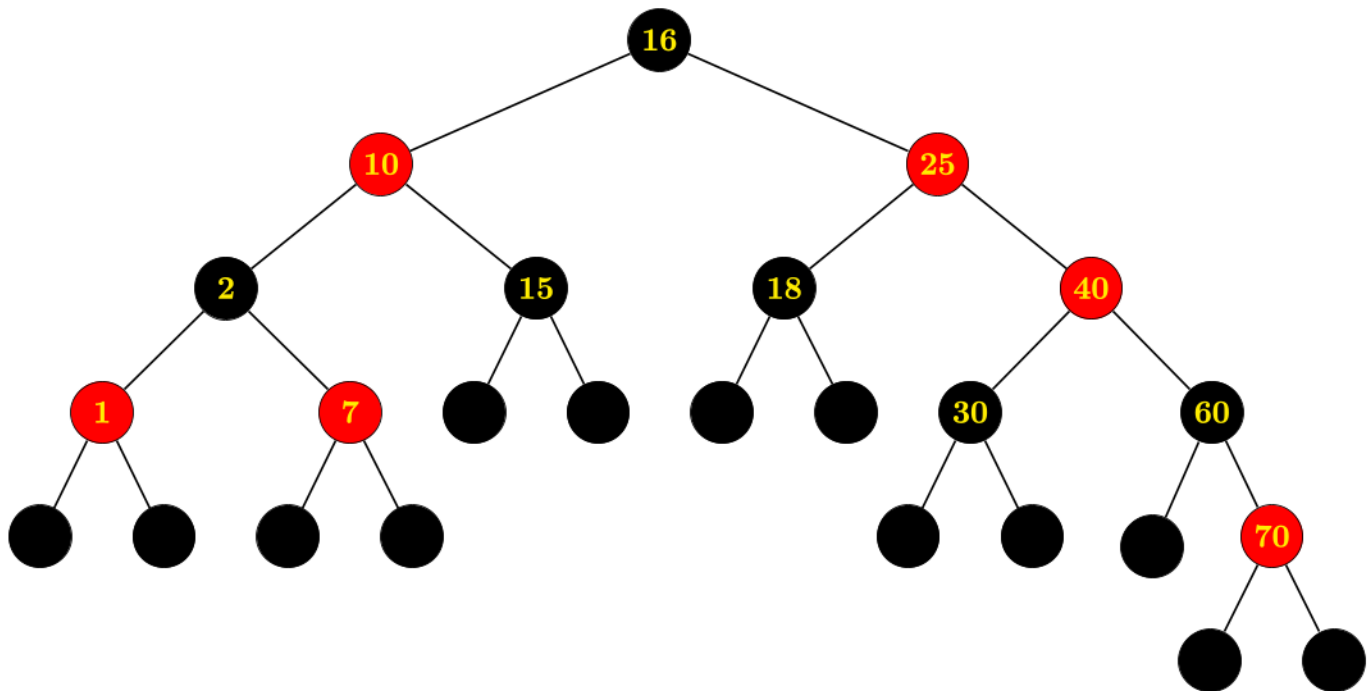


Violation : **RED Node Property Violation !!!**

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? FALSE

Recolor (30,60 BLACK)(40 RED); 40 is new node

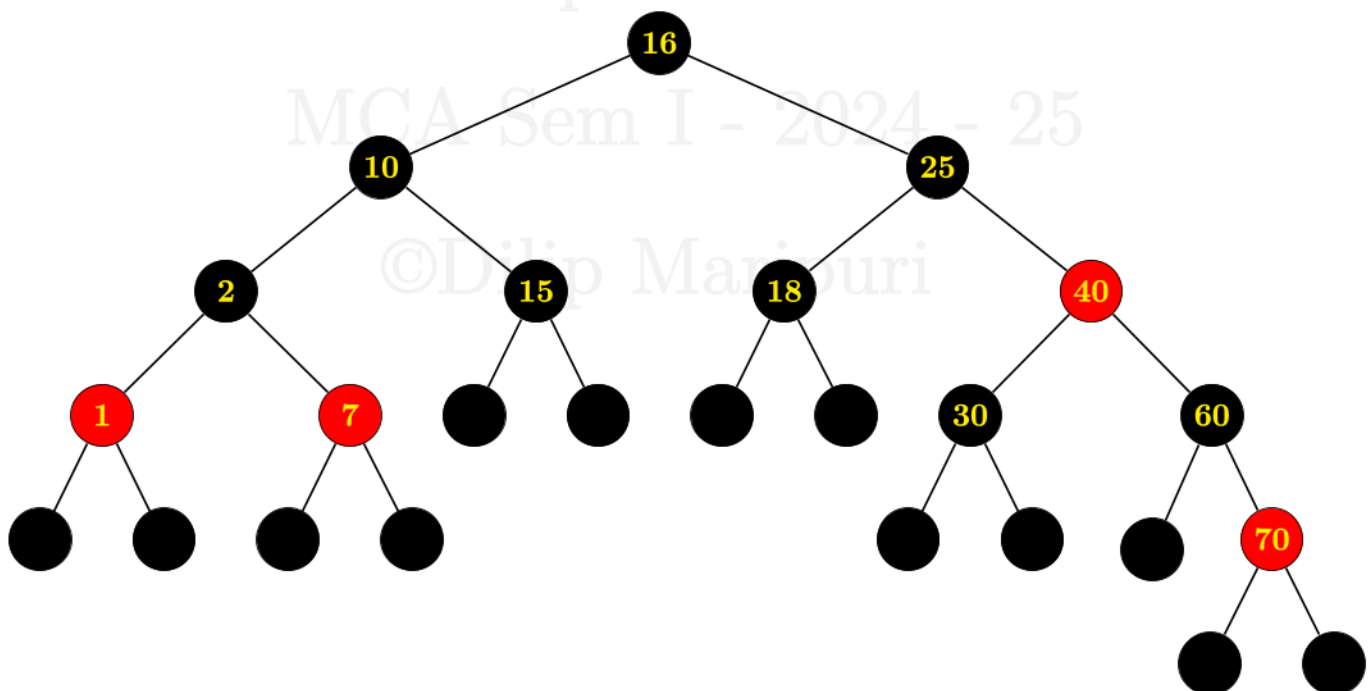


Violation : **RED Node Property Violation !!!**

Color(Parent) is RED ? TRUE

Color(uncle) is BLACK ? FALSE

Recolor (10,25 BLACK)(16 RED); 16 is the root node. So Recolor it Black



1. BLACK Height Property : **Satisfied**

2. RED Node Property : **Satisfied**

3. ROOT Node Property : **Satisfied**